DEVOPS UNIT 3

MINI PROJECT DOCUMENTATION

Enterprise Smart Library Management Platform

A DevOps-oriented case study covering application architecture, automated testing, continuous integration, containerization, and Kubernetes deployment.

Submitted by	Mahesh Bhosale
PRN	2125UDSM1115
Branch	Artificial Intelligence and Data Science
Institution	Sanjivani University
Subject	DevOps Mini Project
Project Topic	Enterprise Smart Library Management Platform

Prepared as a documentation-based mini project report from the supplied project guide. No live deployment or practical execution is claimed in this report.

Table of Contents

Section	Contents
01	Abstract and Project Overview
02	Problem Statement and Objectives
03	Proposed System and Key Features
04	Technology Stack and Repository Structure
05	Application Architecture and API Design
06	Testing Strategy and Quality Assurance
07	CI/CD Pipeline with Jenkins
08	Docker Containerization
09	Kubernetes Deployment

10	End-to-End DevOps Workflow
11	Security, Limitations and Future Scope
12	Conclusion and References

Note: This report documents the supplied mini-project solution and its intended engineering workflow. The commands and configurations are described for explanation; this report does not claim that they were executed on a live system.

1. Abstract and Project Overview

The Enterprise Smart Library Management Platform is a case-study application designed to demonstrate how modern DevOps practices can support the delivery of a small service-oriented web application. The application domain includes a book catalogue, registered users, and borrowing and returning workflows. The supplied implementation guide describes a Python Flask application with REST endpoints, automated tests, a Jenkins pipeline, a Docker image, and Kubernetes manifests.

The purpose of the mini project is to connect application development with repeatable build, test, package, and deployment processes. Rather than treating deployment as a separate manual activity, the proposed workflow makes code changes traceable through Git, verifies them with automated tests, packages the application into a container, and describes the desired runtime state through Kubernetes resources.

Project Summary

Item	Description
Project name	Enterprise Smart Library Management Platform
Project category	DevOps / CI-CD / Containerization / Orchestration
Application framework	Python 3.11 with Flask 3.0
Testing	Pytest and pytest-cov
CI automation	Jenkins Declarative Pipeline
Containerization	Docker Engine and Dockerfile
Orchestration	Kubernetes manifests on Minikube
Primary objective	Document an automated, repeatable path from source control to deployment

Scope of this Document

This report summarizes the proposed solution from the supplied Word document. It explains the architecture, core functionality, repository organization, quality checks, and deployment workflow. It is a documentation deliverable; no claim is made that AWS, Jenkins, Docker, or Kubernetes resources have been provisioned as part of preparing this report.

2. Problem Statement and Objectives

Problem Statement

A library needs a consistent way to maintain book information, manage user records, and track book borrowing and returns. In a software delivery context, the application also needs a dependable process for testing changes and deploying the same application in a predictable environment. If testing and deployment are handled manually, errors can be missed, environments can differ, and releases can become difficult to repeat.

The proposed solution combines a modular library-management API with an automated DevOps workflow. It demonstrates how source control, automated testing, CI pipeline stages, container packaging, and Kubernetes deployment can reduce avoidable manual effort and improve traceability.

Objectives

- Describe a modular Flask application for users, books, and loans.
- Organize source code and tests in a consistent Git repository structure.
- Use automated unit and API tests to verify important business rules.
- Define a Jenkins pipeline that checks out source code and runs quality checks.
- Package the application using Docker for portable execution.
- Describe Kubernetes Deployment and Service resources for controlled application rollout.
- Document a clear end-to-end workflow and relevant security considerations.

Expected Outcomes

The expected outcome is a documented design that links the application lifecycle from planning and source control to testing, containerization, deployment, and monitoring. The approach can serve as a foundation for a larger academic project or a production-style learning exercise.

3. Proposed System and Key Features

Functional Areas

Module	Purpose	Representative behaviour
User management	Maintain library user records and roles.	Register or look up a user; prevent duplicate user IDs.
Book catalogue	Maintain searchable book metadata and copy counts.	Add books and search by title, author, or ISBN.
Borrowing	Record active loans and update available inventory.	Check user and book existence, quota, and stock before borrowing.
Returns	Close an active loan and restore inventory.	Mark the loan as returned and increment available copies.
Health endpoint	Provide a simple service health response.	GET /api/health returns a status payload.
Automated tests	Check business rules and API behaviour.	Verify successful operations and expected error cases.

Business Rules Described in the Source Guide

- Required user and book fields must be provided.
- Duplicate user or book identifiers are rejected.
- Users cannot be deleted while they have active loans.
- Students are limited to three active loans; faculty members are allowed up to five.
- A book cannot be borrowed if no copies are available.

- A return operation must match an active loan for the specified user and book.

The supplied implementation uses a thread-safe in-memory data store seeded with example users and books. This makes the case study easy to understand, but the data is not persistent across application restarts. A production system would normally use a durable database.

4. Technology Stack and Repository Structure

Technology	Role in the project
Python 3.11	Application language and runtime.
Flask 3.0	Lightweight web framework for REST endpoints.
Pytest / pytest-cov	Automated tests and coverage reporting.
Flake8	Static style and lint checks.
Git / GitHub	Version control, collaboration, and source hosting.
Jenkins	Pipeline automation for repeatable CI stages.
Docker	Builds a portable image containing the application and dependencies.
Kubernetes / Minikube	Manages replicas, health checks, and service access in a cluster.
Gunicorn	Production-style WSGI server for running the Flask app.

Repository Layout

```
library-management-platform/
■■■ app/
■   ■■■ __init__.py
■   ■■■ routes/api.py
■   ■■■ services/library_service.py
■   ■■■ models/
■   ■■■ templates/
■■■ database/db.py
■■■ tests/
■   ■■■ test_user_service.py
■   ■■■ test_book_service.py
■   ■■■ test_borrow_return.py
■   ■■■ test_api_endpoints.py
■■■ k8s/
■   ■■■ deployment.yaml
■   ■■■ service.yaml
■■■ Jenkinsfile
■■■ Dockerfile
■■■ .dockerignore
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

The supplied guide separates work into feature branches for foundation, core application, automated tests, Jenkins CI, Dockerization, and Kubernetes orchestration. The main branch is intended to contain reviewed, stable integration.

5. Application Architecture and API Design

The application is organized into layers so that API request handling, business rules, and data storage are not all mixed into one file. The Flask application factory creates the application and registers an API blueprint. Route handlers receive HTTP requests, call the service layer, and return JSON responses. The service layer applies the library's business rules and uses the database module to access in-memory records.

Logical Architecture

Layer	Responsibility
Client / API consumer	Sends HTTP requests and receives JSON responses.
Flask API routes	Maps HTTP methods and URL paths to application operations.
Library service layer	Enforces user, inventory, quota, borrow, and return rules.
Data store	Stores users, books, and loan records in memory for the case study.
Quality pipeline	Runs tests and static checks before the image is packaged.
Runtime layer	Docker provides the application image; Kubernetes manages replicas and access.

Representative API Endpoints

Endpoint	Method	Purpose
/api/health	GET	Return application health and version information.
/api/users	POST	Register a new user.
/api/users/	GET	Retrieve a user by ID.
/api/books	GET	List books; optional query parameter supports search.
/api/books	POST	Add a book to the catalogue.
/api/borrow	POST	Create a loan after validating business rules.
/api/return	POST	Return a previously borrowed book.

Error Handling

Expected validation failures are represented as HTTP error responses with a JSON error message. This gives API clients a consistent way to distinguish successful operations from invalid input or unavailable records. In a production implementation, additional request-schema validation, authentication, authorization, structured logging, and persistence would be needed.

6. Testing Strategy and Quality Assurance

Automated tests are an essential part of the proposed CI workflow. They provide repeatable evidence that important rules remain correct as the code changes. The supplied guide includes service-level tests and Flask test-client tests for API behaviour.

Test category	Examples of checks	Value
User service tests	User creation, duplicate ID rejection, deletion restrictions.	Protects user-record rules.
Book / inventory tests	Catalogue operations and available-copy handling.	Reduces inventory consistency errors.
Borrow / return tests	Borrowing decrements stock; returning restores stock and loan count.	Verifies the full loan lifecycle.
API tests	Health endpoint status and book-list response.	Checks HTTP interface behaviour.
Test category	Examples of checks	Value

Static analysis	Flake8 checks style and common code-quality issues.	Finds maintainability issues early.
Coverage reporting	Pytest coverage output and report artifacts.	Shows which code paths were exercised.

Illustrative Test Commands

```
python -m pip install -r requirements.txt
python -m flake8 app database tests python
-m pytest -v
python -m pytest --cov=app --cov=database --cov-report=term-missing
```

These are representative commands described for the documented workflow. They are included for clarity and are not evidence that the commands were run during preparation of this PDF.

Quality Gate

The CI pipeline should stop when linting or tests fail. A failed quality gate prevents an unverified version from progressing to packaging or deployment. The supplied guide mentions coverage thresholds as an example; the project team should configure and justify the exact threshold rather than treating an example percentage as an automatically verified result.

7. CI/CD Pipeline with Jenkins

Jenkins automates the repeatable steps that validate a code change. The supplied project guide describes a Declarative Pipeline that checks out source code, prepares dependencies, performs linting and testing, generates reports, and then proceeds to packaging when the checks pass.

Pipeline Stages

Stage	Main activity	Expected evidence
Checkout	Fetch the requested repository revision from Git.	Commit and branch used by the build.
Environment setup	Prepare Python environment and install pinned requirements.	Dependency installation log.
Lint	Run Flake8 against application and test files.	Clean lint result or actionable findings.
Test	Execute Pytest unit and API tests.	Test summary and JUnit-style report.
Coverage / quality gate	Generate coverage report and evaluate configured threshold.	Coverage summary and pass/fail decision.
Package	Build the Docker image after quality checks pass.	Image tag and build output.
Post actions	Publish reports and notify about success or failure.	Build status and archived artifacts.

Why Pipeline as Code?

Keeping the pipeline definition in the repository makes the process reviewable, versioned, and reproducible. It also helps a team see how code changes affect build and release behaviour. Pipeline credentials should be stored in Jenkins credentials management rather than committed to source files.

```
pipeline
{     agent any
stages
{         stage
('Checkout')
{ steps
{ checkout
scm } }
stage('Install
Dependencies')
{             s
teps { sh
'python -m pip
install -r
requirements.tx
t' }         }
stage('Lint')
{             s
teps { sh
'python -m
flake8 app
database
tests' }
}
stage('Test')
{             s
teps { sh
'python -m
pytest -
v' }         }
    }
}
```

The snippet is a simplified illustration of the CI stages, not a replacement for the complete Jenkinsfile specified by the source guide. Packaging, report publishing, and deployment stages should be added according to the target environment.

8. Docker Containerization

Docker packages the application and its required runtime dependencies into an image. This reduces differences between developer, test, and deployment environments. The supplied guide calls for a Dockerfile, a .dockerignore file, pinned dependencies, and a production-style process using Gunicorn.

Containerization Flow

- Start from a suitable Python base image.
- Install only the required Python packages from requirements.txt.
- Copy the application modules into the image.
- Configure the container to run the application using a WSGI server.
- Build and tag the image with a version or commit identifier.
- Run a container locally and check the health endpoint before release.

Illustrative Docker Commands

```
docker build -t library-platform:v1.0.0 .
docker run --rm -p 5000:5000 library-platform:v1.0.0
# In a separate terminal: curl
http://localhost:5000/api/health
```

The .dockerignore file should exclude virtual environments, Python caches, Git metadata, test caches, and local secrets. Production images should use maintained base images, avoid running as root where practical, and be rebuilt regularly to receive security updates.

Benefits

Containerization improves portability, reduces environment drift, and gives the deployment platform a consistent artifact to run. It does not automatically make an application secure; image scanning, dependency updates, secret management, and runtime restrictions remain important.

9. Kubernetes Deployment

Kubernetes manages the desired state of a containerized application. The project guide provides a Deployment with three replicas and health probes, along with a NodePort Service configured to use port 30080. Minikube is the local cluster environment used in the case study.

Deployment and Service Roles

Resource	Role in this project
Deployment	Declares the application image, replica count, container port, and health probes. Kubernetes works to maintain the desired number of Pods.
Pod	Runs an instance of the application container.
Service	Provides a stable access point for the selected Pods.
NodePort	Exposes the service through a port on the cluster node; the guide specifies 30080.
Minikube	Provides a single-node Kubernetes environment suitable for a learning or demonstration setup.

Illustrative Kubernetes Commands

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get deployments kubectl get
pods kubectl get services
kubectl rollout status deployment/library-platform
```

The exact Deployment name must match the name defined in the supplied manifest. The command above uses a representative name and should be aligned with the actual YAML before execution.

Health Checks and Self-Healing

Readiness probes help Kubernetes decide when a Pod can receive traffic, while liveness probes help detect a container that is no longer functioning correctly. A Deployment controller creates replacement Pods when the observed state differs from the declared desired state. These mechanisms improve resilience, but they do not replace application-level monitoring or careful rollout validation.

10. End-to-End DevOps Workflow

Step	Activity	Purpose
1. Plan	Define requirements, acceptance criteria, and security needs.	Keep delivery aligned with user needs.
2. Develop	Implement a small change in a feature branch.	Enable focused and reviewable work.
3. Version	Commit and push changes to GitHub.	Provide traceability and collaboration.
4. Integrate	Jenkins checks out the revision and runs lint/tests.	Detect defects early.

5. Package	Build a versioned Docker image.	Create a portable release artifact.
6. Deploy	Apply Kubernetes manifests to Minikube or a target cluster.	Run the application under orchestration.
7. Verify	Check Pods, service status, and health endpoint.	Confirm that the declared deployment is healthy.
8. Improve	Review logs, test results, and feedback.	Feed operational learning into future changes.

The lifecycle is iterative: findings from tests, deployment, and monitoring inform future changes. Automation improves repeatability, while code review and operational feedback help keep the process reliable.

11. Security, Limitations and Future Scope

Security Considerations

- Store credentials in a secrets manager or CI credential store; never commit them to Git.
- Use least-privilege access for Jenkins agents, container registries, and Kubernetes identities.
- Pin and regularly review dependency versions; scan dependencies and container images.
- Use HTTPS and authentication/authorization for any production-facing API.
- Restrict Kubernetes Service exposure and use network controls appropriate to the environment.
- Protect logs and reports from accidentally including credentials or personal data.

Current Limitations of the Case Study

- The documented data store is in memory, so records do not survive application restarts.
- The supplied sample is intended for learning and does not describe a full production identity system.
- A local Minikube cluster is useful for demonstrations but does not represent a highly available production cluster.
- CI/CD and Kubernetes configurations still require environment-specific credentials, names, and configuration.

Future Enhancements

- Replace the in-memory store with a persistent relational database and migration strategy.
- Add authentication, role-based access control, input schemas, and audit logging.
- Add API documentation, stronger integration tests, and automated security scanning.
- Publish signed/versioned images to a registry and use immutable image tags.
- Introduce monitoring and alerting using suitable metrics, logs, and dashboards.
- Use a managed Kubernetes platform and infrastructure-as-code for production environments.

12. Conclusion and References

The Enterprise Smart Library Management Platform demonstrates how a modest business application can be documented through a DevOps lifecycle. The proposed architecture separates API handling, business logic, and data storage. Automated tests verify key library rules, Jenkins defines repeatable CI stages, Docker packages the application, and Kubernetes describes the intended runtime state.

The main value of the project is the integration of these practices into one traceable workflow. Version control supports collaboration, automated tests help catch defects early, containers reduce environment differences, and Kubernetes provides mechanisms for rollout and recovery. For real production use, persistent data storage, stronger security controls, observability, and deployment-specific validation would be required.

References

- Supplied project guide: Enterprise Smart Library Management Platform — Complete Production Source Code,Automated CI/CD Pipelines & Hands-On Engineering Guide; Document 2: Step-by-Step GitHub Repository, Installation & Deployment Manual.
- Python documentation: https://docs.python.org/3/
- Flask documentation: https://flask.palletsprojects.com/
- Pytest documentation: https://docs.pytest.org/
- Jenkins documentation: https://www.jenkins.io/doc/
- Docker documentation: https://docs.docker.com/
- Kubernetes documentation: https://kubernetes.io/docs/
- Git documentation: https://git-scm.com/doc/